Lab Manual: Practical 4

Course Code: ETCS406

Course Title: Game Development

Program: B. Tech Computer Science and Engineering

Semester: VII

Practical Title: Implement Unity's Physics Components to Simulate Motion and Collisions

Duration: 2 Hours

Lab Type: Simulation & Scripting

Assessment Weightage: Part of CAL 1 (Lab Assignments)

Objective

To understand and apply Unity's physics components to simulate realistic motion and collision behaviors in a 2D or 3D game environment. The goal is to build a simple physics-based interaction such as a bouncing ball or a pinball-style game.

Pre-requisites

- Basic Unity Editor navigation
- Familiarity with Rigidbody, Collider components
- Basic C# scripting knowledge
- Understanding of physics concepts like gravity, force, and collision

Learning Outcomes

By the end of this lab, students will be able to:

- Use Rigidbody and Collider components to simulate physics
- Apply forces and detect collisions using Unity's physics engine
- Write scripts to control object behavior based on physics interactions
- Create a simple physics-based game scenario

Tools & Resources

- Unity Editor (2020.3 LTS or later recommended)
- Visual Studio or any C# IDE

- Unity Asset Store (optional for sprites or prefabs)
- Reference: Unity Learn – Physics Fundamentals

Lab Tasks

Task 1: Scene Setup

- Create a new Unity 2D or 3D project
- Add a ground plane or surface using a sprite or 3D object
- Create a ball GameObject (e.g., circle sprite or sphere)

Task 2: Physics Component Integration

- Attach a **Rigidbody2D** or **Rigidbody** to the ball
- Add a **Collider2D** or **Collider** to both the ball and ground
- Configure gravity, mass, drag, and bounce properties

Task 3: Scripting Motion

- Write a C# script to apply force or impulse to the ball
- Trigger motion using keyboard input or on collision
- Implement bouncing behavior using **Physics Material** (e.g., bounciness)

Task 4: Collision Detection

- Use Unity's collision methods (OnCollisionEnter, OnTriggerEnter)
- Display debug messages or visual effects on collision
- Optional: Add obstacles or walls to simulate a pinball-style layout

Evaluation Criteria

Component	**Marks**
Physics Setup (Rigidbody, Collider)	3
Motion & Force Application	3
Collision Detection & Response	2
Code Quality & Debugging	2
Total	**10**

Tips for Success

- Use **Physics Materials** to fine-tune bounce and friction
- Adjust Rigidbody properties for realistic motion
- Test different force types: AddForce, AddImpulse, Velocity
- Use Unity's **Gizmos** and **Debug.Log** for collision tracking

Suggested Reading & Resources

- *Introduction to Game Development* by Steve Rabin – Chapters on physics simulation
- Unity Learn: Physics and Collisions
- Brackeys YouTube: Unity Physics Tutorial

Submission Guidelines:

- Submit your report in PDF format via the Google Classroom.
- Include screenshots or gameplay references where relevant.